

Rezolvarea Problemei Jocului Mathable

Task 1 - Identificarea poziției în care a fost plasată piesa

Pentru prima cerință, cel mai important pas a fost extragerea matricei din tabla de joc. Pentru a identifica colțurile matricei, am folosit codul de filtrare a culorii. Prin testare am ajuns la concluzia că putem extrage pătratele portocalii din colțurile matricei prin intermediul măștii dacă setăm parametrii astfel:

$$LH = 19 \quad LS = 149$$

$$LV = 151 \quad UH = 92$$

$$US = 255 \quad UV = 255$$

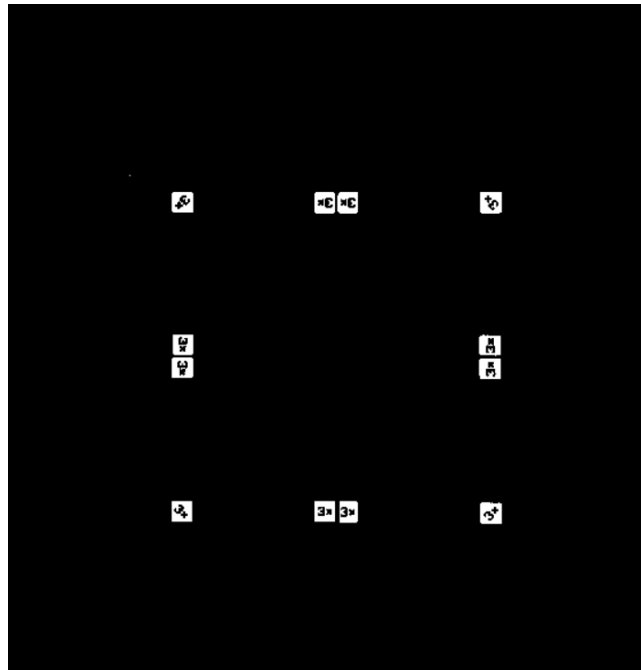


Figure 1: Masca aplicată pentru extragerea colțurilor matricei.

Pentru a identifica coordonatele acestor colțuri, am folosit imaginea cu tabla goală 01.png din folderul de imagini auxiliare. În continuare, am parcurs pixelii măștii și am reținut indexul primului pixel activat de pe coloana din dreapta și stanga și liniile de sus respectiv de jos. Apoi, aceste coordonate pentru decupare au fost aplicate tuturor pozelor cu table de joc.

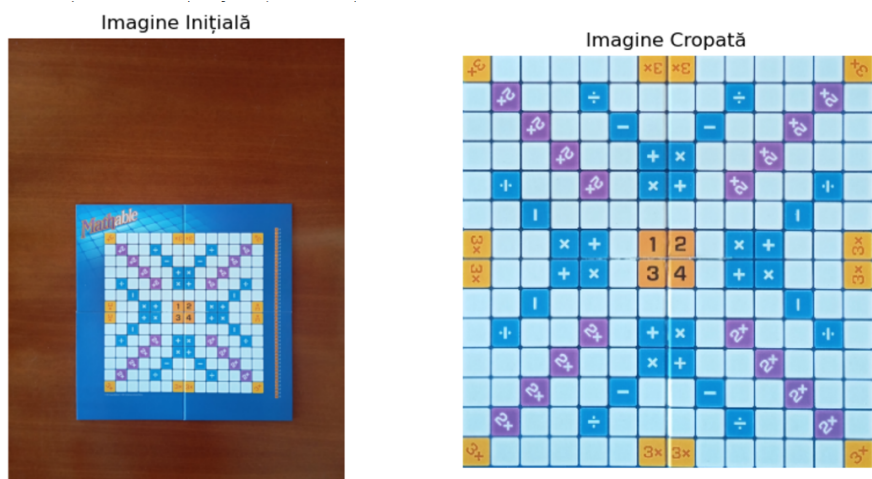


Figure 2: Tabla decupată.

Următoarea funcție folosită a fost pentru evidențierea pieselor de joc. Am folosit următorii parametri:

$$LH = 9 \quad LS = 0$$

$$LV = 180 \quad UH = 90$$

$$US = 71 \quad UV = 255$$

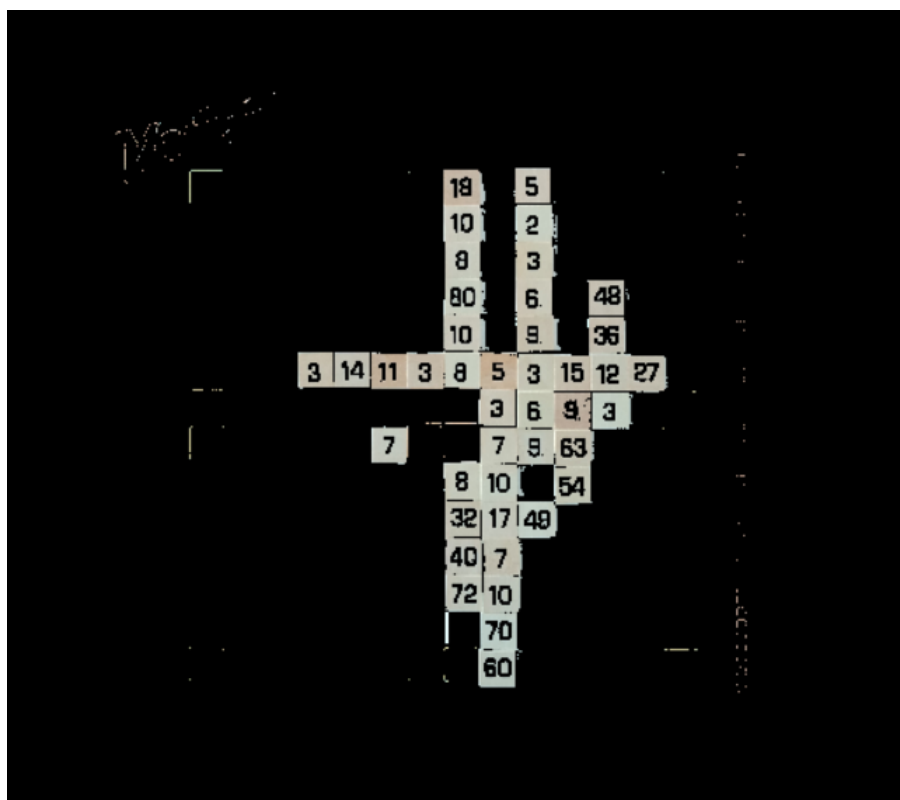


Figure 3: Masca aplicată pentru evidențierea pieselor.

Am iterat prin pozele cu rundele jocului, memorând poza precedentă `prev_image` și cea curentă `current_image`. După ce am decupat centrul tablei și am evidențiat piesele de joc, am apelat funcția principală `detect_piece_on_board`. Am împărțit tabla în 14 pătrate de dimensiuni egale și am utilizat funcția `calculate_diff_matrix` pentru a calcula diferențele

dintre pătratele din imaginea curentă și cea precedentă. Pentru fiecare pereche de pătrate, am aplicat funcția `ssim` din biblioteca `skimage.metrics` pentru a evalua scorul de asemănare, stocând rezultatele într-o matrice. Ulterior, cu ajutorul funcției `find_max_diff`, am identificat pătratul cu cea mai mare diferență, concluzionând că acolo a fost plasată piesa nouă.

Task 2 - Identificarea numărului de pe piesă

Pentru identificarea numărului, am folosit tehnica *template matching*. Astfel, din funcția principală vom apela funcția în care determinăm cel mai bun template pentru piesa.

Listing 1: Funcția pentru procesarea template-urilor.

```
def template_matching(area):
    clean_area = crop_image(area)
    square_cropped = crop_margins(clean_area)
    binarized = binarize_image(square_cropped, threshold=127)

    top, bottom, left, right = crop_coordinates(binarized)
    cropped_image = square_cropped[top:bottom, left:right]

    game_piece_filename = piece_matching(cropped_image)
    return game_piece_filename
```

Prima prelucrare aplicată pătratului este eliminarea marginilor pentru a scăpa de pixelii închiși la culoare. Pentru a fi siguri că am eliminat marginile vom scoate conturul numărului din interiorul piesei cu ajutorul mastii obtinute cu parametrii :

$$\begin{array}{ll} LH = 94 & LS = 0 \\ LV = 0 & UH = 226 \\ US = 255 & UV = 47 \end{array}$$



Figure 4: Masca pentru eliminarea marginilor piesei.

Pentru a evita decuparea excesivă a numărului, folosim un buffer de 8 pixeli. Ulterior, binarizăm imaginea decupată și realizăm decuparea finală în jurul conturului piesei în funcția `crop_coordinates`. Această funcție identifică limitele stânga, dreapta, sus și jos unde se află primul pixel întunecat, iar datorită curățării marginilor, pixelii de la graniță ar trebui să core-spundă marginilor exacte ale numărului. După procesare, piesa este trimisă către funcția propriu-zisă de template matching.

Listing 2: Funcția de template matching.

```
def piece_matching(square):
    gray_square = cv.cvtColor(square, cv.COLOR_BGR2GRAY)

    # determinam daca piesa este cifra sau numar
    # in functie de latimea imaginii
    if gray_square.shape[1] > 48:
        square_resized = cv.resize(gray_square, (70, 55))
        folder = 'templates/numbers'
    else:
        square_resized = cv.resize(gray_square, (37, 55))
        folder = 'templates/digits'

    maxi = -np.inf
    best_template = None
    best_template_image = None

    for template_name in os.listdir(folder):
        template_path = os.path.join(folder, template_name)
        img_template = cv.imread(template_path)

        if img_template is None:
            continue

        # aplicam resize pe template in functie
        # de criteriul cifra/numar
        if folder == 'templates/numbers':
            img_template_resized = cv.resize(img_template, (70, 55))
        else:
            img_template_resized = cv.resize(img_template, (37, 55))

        img_template_gray = cv.cvtColor(img_template_resized,
                                         cv.COLOR_BGR2GRAY)

        corr = cv.matchTemplate(square_resized, img_template_gray,
                                cv.TM_CCOEFF_NORMED)
        max_corr = np.max(corr)

        if max_corr > maxi:
            maxi = max_corr
            best_template = template_name
            best_template_image = img_template_gray

    return best_template
```

În folderul de templates am grupat pozele pe cifre și numere de două cifre, pe baza lățimii. Am observat că piesele cu o singură cifră nu depășesc 45 de pixeli în lățime, dar limita

de 48 a funcționat cel mai bine datorită imperfecțiunilor de decupare. Am folosit funcția `matchTemplate` din OpenCV cu parametrul `cv.TM_CCOEFF_NORMED`, care a dat cele mai bune rezultate. La final, se returnează template-ul cel mai apropiat de imaginea inițială.



Figure 5: Piesa procesată și template-ul găsit.

Task 3 - Calculul scorului pe runde

După identificarea numărului de pe piesă, revenim în funcția `detect_piece_on_board`. Pentru calculul scorului, vom verifica dacă pe poziția identificată există un bonus de x2 sau x3. După ce stabilim valoarea piesei, o vom adăuga în lista de scoruri. La final, vom apela funcția de salvare pentru a înregistra poziția și piesa în fișierul text corespunzător. În cadrul buclei principale, atunci când ajungem la fișierul `turns.txt`, vom genera fișierul `scores.txt` prin intermediul funcției `generate_scores_file`.

Listing 3: Funcția pentru generarea fișierului de scoruri.

```
def generate_scores_file(turns_file_path, game_number, scores_list):
    with open(turns_file_path, "r") as turns_file:
        turns_lines = turns_file.readlines()

    with open(f"341_Popeanga_Antonia/{game_number}_scores.txt", "w")
        as scores_file:
            for idx, line in enumerate(turns_lines):
                parts = line.strip().split()
                player, start_turn = parts[0], int(parts[1]) - 1
                if idx < len(turns_lines) - 1:
                    end_turn = int(turns_lines[idx + 1].split()[1]) - 1
                else:
                    end_turn = len(scores_list)

                score_sum = sum(scores_list[start_turn:end_turn])
                scores_file.write
                (f"{player} {start_turn + 1} {score_sum}\n")
```

Astfel, vom face suma listei de scoruri în funcție de runde jucate de cei doi jucători. La final, în bucla principală, vom reseta parametrii pentru a începe procesarea datelor pentru noul joc.